

# Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling

YANNAN NELLIE WU, MIT, USA

PO-AN TSAI, NVIDIA, USA

ANGSHUMAN PARASHAR, NVIDIA, USA

VIVIENNE SZE, MIT, USA

JOEL S. EMER, MIT/NVIDIA, USA

In recent years, many accelerators have been proposed to efficiently process sparse tensor algebra applications (e.g., sparse neural networks). However, these proposals are single points in a large and diverse design space. The lack of systematic description and modeling support for these sparse tensor accelerators impedes hardware designers from efficient and effective design space exploration.

This paper first presents a unified taxonomy to systematically describe the diverse sparse tensor accelerator design space. Based on the proposed taxonomy, it then introduces Sparseloop, the first fast, accurate, and flexible analytical modeling framework to enable early-stage evaluation and exploration of sparse tensor accelerators. Sparseloop comprehends a large set of architecture specifications, including various dataflows and sparse acceleration features (e.g., elimination of zero-based compute). Using these specifications, Sparseloop evaluates a design’s processing speed and energy efficiency while accounting for data movement and compute incurred by the employed dataflow, including the savings and overhead introduced by the sparse acceleration features using stochastic density models.

Across representative accelerator designs and workloads, Sparseloop achieves over 2000× faster modeling speed than cycle-level simulations, maintains relative performance trends, and achieves 0.1% to 8% average error. The paper also presents example use cases of Sparseloop in different accelerator design flows to reveal important design insights.

Additional Key Words and Phrases: accelerator modeling, sparse tensor algebra, neural networks

## 1 INTRODUCTION

Sparse tensor algebra is widely used in many important applications, such as scientific simulations [65], computer graphics [4], graph algorithms [19, 41], and deep neural networks (DNNs) [1, 38]. Depending on the sparsity characteristics of the tensors (e.g., sparsity, distribution of zero locations), sparse tensor algebra can introduce a significant number of *ineffectual computations*, whose results can be easily derived by applying the simple algebraic equalities of  $X \times 0 = 0$  and  $X + 0 = X$ , without reading all the operands or doing the computations [18, 33].

As a result, many performant and energy-efficient sparse tensor algebra accelerators have been proposed to exploit ineffectual computations to reduce data movement and compute [2, 8, 9, 12, 15, 18, 22, 34, 35, 37, 39, 47, 53, 61]. Based on the properties of its target applications (e.g., convolution or matrix multiplication), each accelerator design proposes its unique hardware support. Various accelerators can: propose different architecture topology (e.g., number of storage levels); employ different *dataflows* [8] (i.e., the rules for scheduling data movement and compute in space and time); use different *encoding* to compress sparse tensors (e.g., bitmask encoding); and design different hardware to eliminate operations associated with ineffectual computations (e.g., intersection units). The joint design space for all these hardware mechanisms is therefore large and diverse.

To characterize either a single specific design or many designs as part of design space exploration, hardware designers can benefit from a modeling framework that is:

---

Authors’ addresses: Yannan Nellie Wu, nelliewu@mit.edu, MIT, , Cambridge, MA, USA; Po-An Tsai, poant@nvidia.com, NVIDIA, , Westford, MA, USA; Angshuman Parashar, aparashar@nvidia.com, NVIDIA, , Westford, MA, USA; Vivienne Sze, sze@mit.edu, MIT, , Cambridge, MA, USA; Joel S. Emer, emer@csail.mit.edu, MIT/NVIDIA, , Cambridge, MA, USA.

|                             | Accuracy    | Speed       | Flexibility | Support Sparsity? |
|-----------------------------|-------------|-------------|-------------|-------------------|
| Cycle-Level Design-Specific | Very High   | Slow        | Low         | Yes               |
| General Analytical          | High        | Fast        | High        | No                |
| <b>Our Work</b>             | <b>High</b> | <b>Fast</b> | <b>High</b> | <b>Yes</b>        |

Table 1. Comparison of existing tensor accelerator simulation frameworks with our proposed Sparseloop framework.

- **Flexible:** is capable of modeling a diverse range of potential designs with hardware support for different dataflows, compression encodings, etc.
- **Fast:** produces simulation results quickly. This is particularly important because properly characterizing a specific design requires finding the best schedule, i.e., *mapping*, for a given workload, which generally requires a search of a large mapspace [6, 23, 36].
- **Accurate:** produces simulation results correctly in both mapspace and design space exploration.

However, to the authors’ knowledge, none of the existing modeling approaches for tensor accelerators provide the desired capabilities (Table 1). On the one hand, *cycle-level design-specific models* [2, 8, 9, 18, 30, 35, 37, 39, 47, 60, 62, 64] capture the detailed implementations for their target designs (e.g., memory control signals) and thus are very accurate. However, such models impede users from mapspace exploration due to their slow simulation speed and design space exploration due to their limited parameterization support. On the other hand, analytical models perform mathematical computations to analyze the important high-level characteristics of a class of accelerators and are fast. However, the existing *general analytical models* are only flexible for dense accelerator designs [21, 24, 29, 36, 43, 44, 48, 55, 58, 66], i.e., they do not reflect the impact of sparsity-aware acceleration techniques, resulting in inaccurate modeling.

To address the limitations of existing work, we present Sparseloop<sup>1</sup>, the *first analytical modeling framework* for fast, accurate, and flexible evaluations of sparse (and dense) tensor accelerators, enabling early-stage exploration of the large and diverse sparse tensor accelerator design space. Table 1 compares Sparseloop to existing simulation frameworks.

**This work makes the following key contributions:**

- (1) To systematically describe the large and diverse sparse tensor accelerator design space, we propose a taxonomy to classify the various sparsity-aware acceleration techniques into three *sparse acceleration features (SAFs)*: representation format, gating, and skipping.
- (2) Based on the SAF classification, we propose Sparseloop, an analytical modeling framework for tensor accelerators.

- To both faithfully reflect workload data’s impact on accelerator performance and ensure simulation speed, Sparseloop performs analysis based on statistical characterizations of nonzero value locations in the tensors.
- To keep the modeling complexity tractable and allow support for emerging workloads/designs, Sparseloop splits its modeling process into discrete steps, each of which focuses on evaluating a distinct design aspect (e.g., dataflow, sparse acceleration features). This decoupling allows modeling of both dense and sparse designs in one infrastructure.

- (3) With representative accelerator designs and workloads, we show that Sparseloop is fast, accurate, and flexible:

- Sparseloop runs more than 2000× faster than a cycle-level simulator.
- Sparseloop maintains relative performance trends and achieves 0.1% to 8% average error across designs.

<sup>1</sup>Sparseloop is open-source and publicly available at [Sparseloop website](#) [56].

- Sparseloop allows comparison of designs with different dataflows and sparse acceleration features, running workloads with various sparsity characteristics.
- Sparseloop can reveal design insights during accelerator design flows. Our case studies demonstrate Sparseloop’s flexibility to quickly compare and explore diverse designs with different architectures, dataflows, and SAFs running various workloads.

## 2 BACKGROUND AND MOTIVATION

In this section, we illustrate the complexity of describing and evaluating the sparse tensor accelerator design space.

### 2.1 Large and Unstructured Design Space

Sparse tensor accelerators often employ different dataflows to exploit data reuse across multiple storage levels and feature various sparsity-aware acceleration techniques to eliminate data storage for zeros and *ineffectual operations* (IneffOps), *i.e.*, arithmetic operations and storage accesses associated with ineffectual computations. The vast number of potential design choices lead to a large and diverse design space.

Nonetheless, there is little structure in the design space for sparse tensor accelerators, as each prior design uses different terminology to describe a point in the design space. We present the design decisions made by representative designs to show the lack of uniformity in their architecture proposals.

For example, Eyeriss [8] uses a *row-stationary* dataflow, a *RLC* encoding for data stored in DRAM, and storage and compute units that stay idle for IneffOps. With the same dataflow, Eyeriss V2 [9] employs a *compressed sparse column* encoding for both on-chip and DRAM data, and avoids spending cycles for IneffOps by performing intersections near the compute units. SCNN [37] also uses a similar intersection-based acceleration, but features a *PlanarTiled-InputStationary-CartesianProduct* dataflow and *compressed-sparse-block* encoding. ExTensor [18] proposes a *hybrid* dataflow and a *hierarchical* encoding. It introduces the *hierarchical-elimination* acceleration technique, which aggressively eliminates IneffOps at multiple storage levels long before data reaches compute. Dual-side sparse tensor core (DSTC) [53] uses an *output-stationary* dataflow and *two-level BitMap* encoding. It designs an *operand-collector* hardware unit tailored to its dataflow to provide enough bandwidth after elimination of IneffOps.

Since different accelerators propose different sets of implementation choices, often described in design-specific naming conventions, it is challenging for designers to have a systematic understanding of the proposed dataflow and acceleration techniques in the design space, let alone a modeling framework to compare these designs systematically.

### 2.2 Sparsity Impacts Design Behavior

Evaluating the complex design space of sparse tensor accelerators is further complicated by the impact of the tensor *sparsity characteristics*, which include the density (*i.e.*, percentage of nonzero values in each tensor, 1–sparsity) and the locations of nonzero values in each tensor.

To demonstrate this entanglement, we compare two designs supporting different data representations. For simplicity, both accelerators employ the same dataflow:

**(1) Bitmask (Eyeriss-like):** The first design supports bitmask encoding to represent sparse operand tensors. Bitmask uses a single bit to encode whether each value is nonzero or not. In each cycle, the design uses each bit to decide whether its storage and compute units should stay idle to save energy, but it does not improve processing speed.<sup>2</sup>

<sup>2</sup>Of course, there exist other designs that use bitmasks to save both energy and time [15, 62]

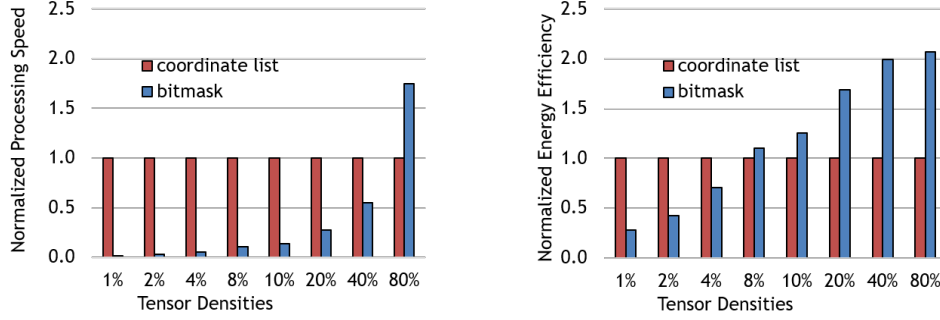


Fig. 1. Processing speed and energy efficiency of architectures with different data representation support running sparse matrix multiplication workloads. Design behavior is dependent on data representations and tensor densities.

(2) **Coordinate list (SCNN-like)**: The second design employs a coordinate list encoding [11, 50], which indicates the location of each nonzero value via a list of its coordinates (*i.e.*, the indices in each dimension). Since the coordinate information directly points to the next effectual computation, the design only spends cycles on effectual operations, thus saving both energy and time.

In Fig. 1, we compare the processing speed and energy efficiency of the two designs running sparse matrix multiplication workloads of different densities. As shown in Fig. 1, the best design choice is a function of the input density. More specifically, since the bitmask-based design does not improve processing speed, with low-density tensors, *bitmask* always runs slower than *coordinate list*. However, since *coordinate list* needs to encode the exact coordinates with multiple bits, it incurs more significant encoding overhead per nonzero value. As the tensors become denser, *coordinate list* leads to lower energy efficiency and/or processing speed. This trend has also been observed in Sigma [39].

Even just varying the input tensor density, we already see non-trivial interactions between the benefits introduced by eliminated IneffOps and compressed sparse tensors, and the overhead introduced by extra encoding information. A more involved case study in Sec. 7.1 will further showcase the complex interactions between dataflows, sparsity-aware acceleration techniques, and workload sparsity characteristics, illustrating the importance of co-designing various design aspects. Thus, for hardware designers to efficiently explore the trade-offs of various design decisions, there is a strong need to have a *fast modeling framework that, in addition to evaluating various dataflows, recognizes the impact of the different acceleration techniques and tensor sparsity characteristics on processing speed and energy efficiency*.

**Our proposal:** To address these two issues, we first introduce a new classification of the various sparsity-aware acceleration techniques (Sec. 3), which unifies how to qualitatively describe these techniques in a systematic manner. Leveraging this taxonomy, we then propose an analytical modeling framework, Sparseloop, which quantitatively evaluates the diverse sparse tensor accelerator designs (Sec. 4 to Sec. 5.4). We show that Sparseloop is fast, accurate, and flexible in Sec. 6.2, 6.3, and 7.

### 3 DESIGN SPACE CLASSIFICATION

The first step toward a systematic modeling framework is to have a unified taxonomy to describe various sparse tensor accelerators. We propose a new classification framework that simplifies how to describe a specific design in the complex design space. We then demonstrate how prior designs can be described in a straightforward manner.

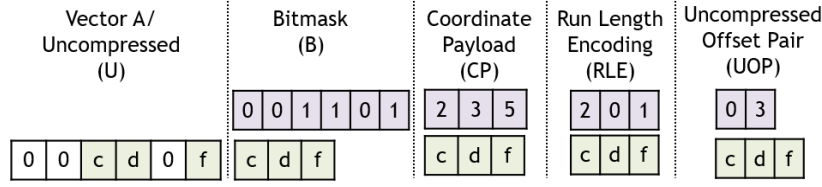


Fig. 2. Example representation formats of a vector A. Purple vectors refer to metadata used to identify the original locations of the nonzero values.

### 3.1 High-Level Sparse Acceleration Features

To systematically describe sparse tensor accelerators in the design space, we classify common sparsity-aware acceleration techniques into three orthogonal high-level categories:

- Representation format
- Gating IneffOps
- Skipping IneffOps

We call each category a *sparse acceleration feature* (SAF).

**3.1.1 Representation Format.** Representation format refers to the choice of encoding the locations of nonzero values in the tensor. To describe a representation format, we adopt a hierarchical expression that combines multiple per-dimension formats, similar to [11, 26, 50].

As shown in Fig. 2, we introduce several commonly used per-dimension formats with an example 1D tensor, *i.e.*, a vector. The most basic format is *Uncompressed (U)*, which represents the tensor with its exact values, thus directly showing the locations of nonzero values. *U* is identical to the original vector. However, to save storage space, and thus implicitly save energy (and time) associated with zero value accesses, sparse tensor accelerators tend to employ *compressed formats*, which represent a tensor with only nonzero values and some additional information about their original locations or *coordinate* [11, 26, 50]. We call this information *metadata*. We introduce four per-dimension compressed formats<sup>3</sup>.

- *Coordinate Payload (CP)*: the coordinates of each nonzero value are encoded with multiple bits. The *payloads* are either the nonzero value or a pointer to another dimension. CP explicitly lists the coordinates and the corresponding payloads.
- *Bitmask (B)*: a single bit is used to encode whether each coordinate is nonzero or not.
- *Run Length Encoding (RLE)*: multiple bits are used to encode the run length, which represents the number of zeros between nonzeros (*e.g.*, an  $r$ -bit run length can encode up to a  $2^r - 1$  run of zeros).
- *Uncompressed Offset Pairs (UOP)*: multiple bits are used to encode the start (inclusive) and end (noninclusive) positions of nonzero values.

As shown in Table 2, full tensor representation formats can be described by combining the per-dimension formats in a hierarchical fashion. For example, CSR (compressed sparse row) [42] can be described by *UOP-CP*: Top level UOP encodes the start and end locations of the nonzeros in each row; bottom level CP encodes the exact column coordinates and its associated nonzeros. A format can also split and/or flattened tensor dimensions (*e.g.*, 2D COO flattens multiple dimensions into one dimension represented by CP with tuples as coordinates). We use a superscript to indicate the number of flattened dimensions.

<sup>3</sup>Of course, many more per-dimension formats exist and can be incorporated modularly into Sparseloop.

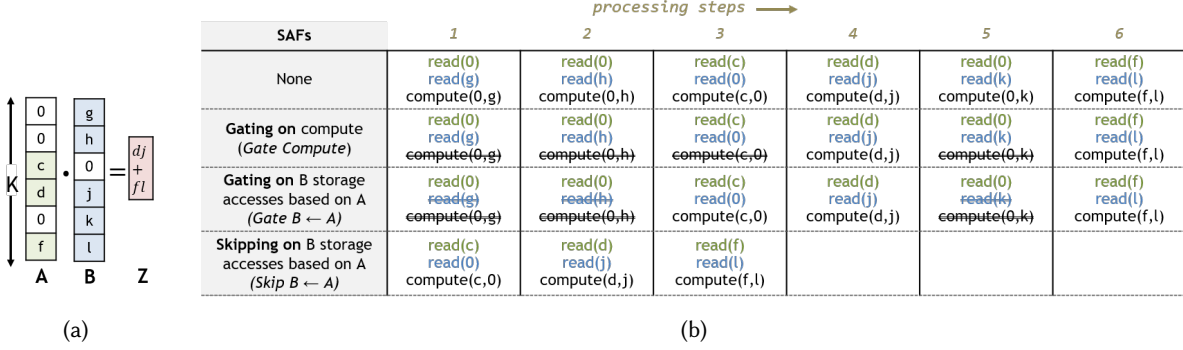


Fig. 3. (a) Sparse dot product workload. (b) Example ways of processing the example workload. 1st row: baseline processing without SAFs; 2nd row: Gating applied to compute; 3rd row: Gating applied to B reads based on A's values; 4th row: Skipping applied to B reads based on A's values.

| Example Classic<br>Representation Formats | Hierarchical<br>Description |
|-------------------------------------------|-----------------------------|
| Compressed Sparse Row (CSR) [42]          | UOP-CP                      |
| 2D Coordinate List (COO) [51]             | CP <sup>2</sup>             |
| Compressed Sparse Block (CSB) [5]         | UOP-CP-CP                   |
| 3D Compressed Sparse Fibers (CSF) [46]    | CP-CP-CP                    |

Table 2. Example representation formats and their hierarchical description based on per-dimension formats.

**3.1.2 Gating.** Gating exploits the existence of IneffOps by letting the storage and compute units stay idle during the corresponding cycles. As a result, it saves energy but does not change processing speed. Gating can be applied to both compute and storage units in the architecture.

We use the dot-product workload in Fig. 3a to illustrate the impact of gating. Each row of Fig. 3b corresponds to a specific SAF implementation, each column is a processing step, and each cell lists the operations happening at the step. The first row presents the baseline processing without any SAFs applied, so it performs all IneffOps and takes six steps to complete.

The second row in Fig. 3b shows the result of applying gating to compute units, *Gate Compute*. The compute unit checks whether operands are zeros and stays power-gated if at least one operand is zero.

When gating is applied to storage units, it can be based on one of two approaches:

(1) **Leader-follower intersection** checks one operand, and if this operand is zero, it avoids accessing the other operand. We call the checked operand the *leader* and the operand with gated access the *follower*. In our classification, gating based on leader-follower intersection is represented by an arrow that points from the leader to the follower, i.e., *Gate Follower* ← *Leader*. The third row in Fig. 3b shows *Gate B* ← *A*. Note that this approach may not eliminate all IneffOps (e.g., step three in the example), and the savings introduced depend on the leader operand's sparsity characteristics.

(2) **Double-sided intersection** checks both operands (usually just via their associated metadata), and if either of them is zero, it does not access either operands' data. Double-sided intersection is represented with a double-sided



arrow that points to both operands, i.e.,  $Operand0 \leftrightarrow Operand1$ . Double-sided intersection eliminates all IneffOps but may require more complex hardware.

In addition to reducing storage accesses, gating applied to a storage unit also leads to *implicit* gating of the compute unit connected to it (e.g., step one in the third row of Fig. 3b), as the compute unit can now use the check for the storage unit to power-gate itself.

**3.1.3 Skipping.** Skipping refers to exploiting IneffOps by not spending the corresponding cycles. Since skipping directly skips to the next effectual computation, it saves both energy and time. Similar to gating, skipping can be applied to both the compute and storage units.

When skipping is applied to compute units, the compute units directly look for the next pair of operands until it finds effective computations to perform. When skipping is applied to storage units, it can also be based on leader-follower intersection or double-sided intersection. However, instead of letting the storage stay idle, with skipping applied, cycles are only spent on effectual accesses. The last row in Fig. 3b shows an example implementation of skipping B reads based on A’s values ( $Skip\ B \leftarrow A$ ). Similar to gating, a leader-follower implementation of skipping can still introduce some IneffOps, and skipping at storage can lead to implicit skipping at the compute units. Since skipping needs to quickly locate the next effectual operation to skip to, it usually requires more complex hardware than gating does (e.g., ExTensor’s intersection unit implements smart look-ahead optimizations to locate effectual operations in time [18]). Inefficient implementations can lead to more overhead than savings in time and energy.

| Design         | Workload | Format <sup>†</sup>                            | Gating/Skipping                                                                                    |
|----------------|----------|------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Eyeriss [8]    | DNN      | offchip: I/O: B-RLE W:U<br>onchip: I: UB O/W:U | Innermost Storage : $Gate\ W \leftarrow I, Gate\ O \leftarrow I$                                   |
| Eyeriss V2 [9] | DNN      | I/W: B-UOP-CP O:U                              | Innermost Storage : $Skip\ W \leftarrow I, Skip\ O \leftarrow I \&W; Gate\ Compute$                |
| SCNN [37]      | DNN      | I/W: B-UOP-RLE O: U                            | Innermost Storage : $Skip\ W \leftarrow I, Skip\ O \leftarrow I \&W; Gate\ Compute$                |
| ExTensor [18]  | MM       | A/B: UOP-CP×5 Z: U                             | All Storage : $Skip\ A \leftrightarrow B, Skip\ Z \leftarrow A \&B$                                |
| DSTC [53]      | MM       | A/B: B-B Z: U                                  | $2^{nd}$ -to-innermost & Innermost Storage : $Skip\ A \leftrightarrow B, Skip\ Z \leftarrow A \&B$ |

Table 3. Summary of representative sparse tensor accelerators described with the proposed SAFs based on tensors from example target workloads. For DNN: I: input activation, W: Weights, O: output activation. For Matrix Multiplication (MM): A,B: operand tensors, Z: result tensor. Note that the designs have different dataflows, which are not listed.

## 3.2 Dataflow is Orthogonal to Sparsity-Aware Acceleration

In addition to the SAFs, dataflow choice is another important decision made by various accelerators [7]. A taxonomy of dataflows for various tensor algebra workloads has already been well studied in existing work (e.g., for DNNs [7, 50], and matrix multiplications [35, 47, 61]).

We make the observation that the dataflow choice is *orthogonal* to the chosen SAFs. Dataflows define the scheduling of data movement and compute in time and space, and SAFs define the actual amount of data that is moved or number of computes performed. As a result, the space of sparse tensor accelerators is the cross product of dataflow choices and SAF choices (further information on how this impacts modeling is in Section 4). Of course, a particular dataflow might mesh well with a specific SAF implementation, leading to an efficient design, while another may not.

## 3.3 Describing Sparse Tensor Accelerators

General accelerator designs often implement multiple SAFs that work well with each other to efficiently improve hardware performance. Fig. 4 illustrates the idea with a simple example, for the same workload in Fig. 3a, Fig. 4

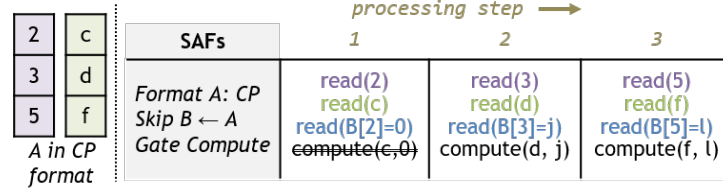


Fig. 4. Example of combining compressed format, skipping, and gating SAFs in one design.

employs a CP representation format for vector A, *Skip B ← A* and *Gate Compute*. By representing A with CP, *Skip B ← A* is implemented by directly reading the appropriate B values based on A’s metadata. Furthermore, by applying *Gate Compute*, Fig. 4 eliminates the compute unit’s IneffOps for cases with nonzero A and zero B.

Realistic sparse tensor accelerators often feature multiple storage levels to exploit data reuse opportunities and a set of spatial compute units for parallel computation. Thus, to systematically describe each design, we need to define the SAFs implemented at each level in the architecture. Based on our proposed classification, Table 3 describes the acceleration techniques of the representative designs introduced in Sec. 2.

For example, SCNN [37], a sparse DNN accelerator, uses a three-level, B-UOP-RLE representation format<sup>4</sup> to compress input activation (IA) and weights (W). In the innermost storage level, i.e., the level closest to the compute units, SCNN performs *SkipW ← IA* and *Skip OA ← IA & W*, where OA refers to output activation. Gating is applied to compute units, i.e., *Gate Compute*, to eliminate leftover IneffOps, similar to Fig. 4’s strategy. ExTensor [18] is an accelerator for general sparse tensor algebra. We use matrix multiplication as an example workload, which involves operand tensors A, B and result tensor Z. ExTensor partitions and compresses tensors with a six-level format and performs *Skip A ↔ B* and *Skip Z ← A & B* at all storage levels.

Thus we hope it is clear how this taxonomy allows the design-specific terminologies in existing proposals to be translated into systematic descriptions. More importantly, it also allows future sparse accelerators to be described accurately and compared *qualitatively* in the same way.

## 4 SPARSELOOP OVERVIEW

The design space taxonomy in Sec. 3 lays the foundation for the modeling methodologies of Sparseloop, an analytical modeling framework that *quantitatively* evaluates the processing speed and energy efficiency of sparse tensor accelerators. In this section, we will discuss the modeling challenges and Sparseloop’s key methodologies to address those challenges.

### 4.1 Modeling Challenges

There are three key challenges associated with ensuring the modeling framework’s speed, accuracy, and flexibility. **Multiplicative factors of the design space.** To faithfully model various sparse tensor accelerator designs, the analysis framework needs to understand the compound impact of their sparsity-specific design aspects (e.g., the diverse SAFs shown in Table 3) together with general design aspects (e.g., architecture topology, dataflow, etc.). Simultaneously modeling the interactions between a considerable number of design aspects incurs high complexity, slowing down the modeling process. Building specific models for each design cannot scale to cover the entire design space, either.

**Tradeoff between accuracy and modeling speed.** High fidelity modeling requires time-consuming sparsity-dependent analysis. Since sparsity characteristics impact a sparse accelerator’s performance, carefully examining the exact data in each tensor could ensure accuracy. However, the downside of actual-data based analysis is that

<sup>4</sup>Some per-dimension formats are applied to split or flattened dimensions.



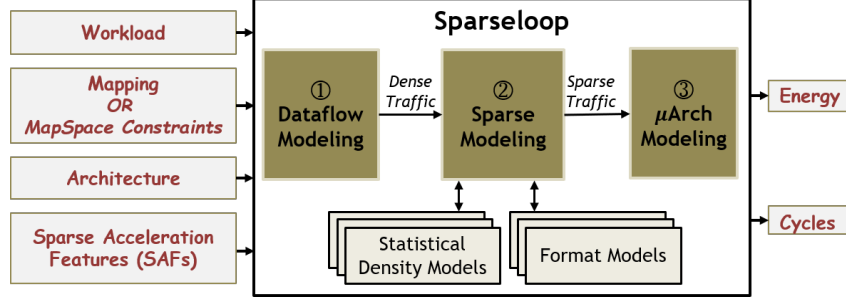


Fig. 5. Sparseloop High-Level Framework.

it can cause intolerable slowdown during mapspace exploration, especially for workloads with numerous and evolving data sets, e.g., DNNs.

**Evolving designs/workloads.** Finally, diverse and constantly evolving designs/workloads require flexibility and extendability in the modeling framework. Since the interactions between the processing schedules and workload data characteristics are convoluted, the framework must be flexible and modularized enough to allow easy extensions for future designs/workloads.

#### 4.2 Sparseloop Solutions to the Challenges

To solve the challenges, Sparseloop makes two important observations for sparse accelerator modeling: (1) the runtime behaviors of sparse accelerators (e.g., number of storage accesses and computes) can be progressively modeled; (2) the sparsity-dependent behavior in sparse accelerators can be statistically modeled with negligible errors.

Based on observation (1), to maintain modeling complexity, Sparseloop performs decoupled modeling of distinct design aspects: Sparseloop evaluates dataflow independent of SAFs, as the storage accesses and computes introduced by the dataflow are irrelevant to how the IneffOps get eliminated; Sparseloop evaluates SAFs independent of micro-architecture, as the number of eliminated IneffOps introduced by the SAFs is orthogonal to the cost of performing each elimination or the savings brought by each eliminated IneffOp. Thus, as Fig. 5 shows, Sparseloop’s modeling process is split into three steps, each with tractable complexity.

- **Dataflow modeling:** analyzes the uncompressed data movement and dense compute, *i.e.*, dense traffic, incurred by the user-specified mapping input.
- **Sparse modeling:** analyzes and reflects the impact of SAFs by filtering the dense traffic to produce sparse data movement and sparse compute, *i.e.*, sparse traffic.
- **Micro-architecture modeling:** analyzes the exact hardware operation cost (e.g., multi-word storage access cost) and generates the final energy consumption and processing speed based on the sparse traffic.

Based on observation (2), Sparseloop enables systematic recognition of the impact of SAFs at the sparse modeling step. To balance accuracy and speed, sparse modeling performs analysis based on *statistical characterizations* of nonzero value locations in workload tensors and their sub-tensors, by leveraging various statistical density models.

Finally, as shown in Fig. 5, to ensure extendability, the sparse modeling step interacts with statistical density models and per-dimension format models as decoupled modules so that these models can be extended to support future sparse workloads and representation formats.

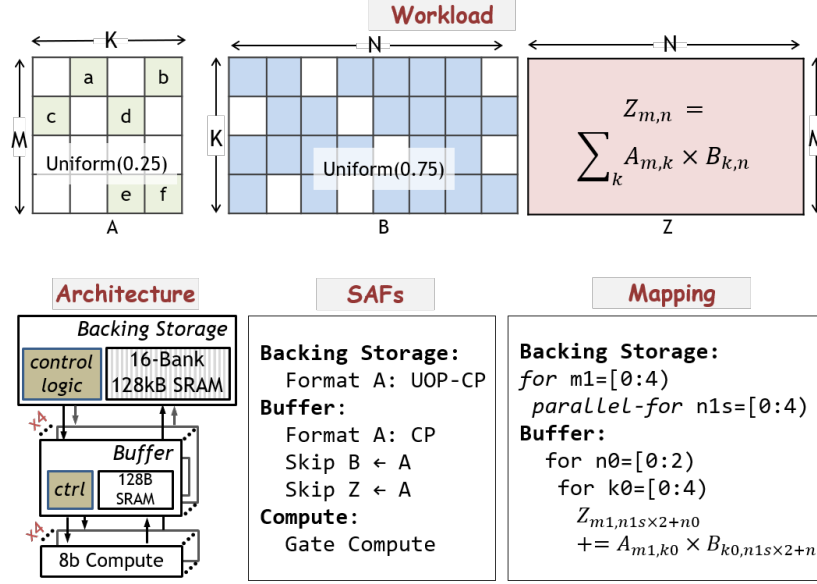


Fig. 6. Example input specifications to Sparseloop. Blank spaces in the workload tensors refer to locations with zeros. The locations of zeros are just for illustrative purposes.

## 5 SPARSELOOP FRAMEWORK

We first discuss the inputs to Sparseloop in Sec. 5.1, and describe the modeling steps in Sec. 5.2, 5.3, and 5.4.

### 5.1 Inputs

As shown in Fig. 5, Sparseloop needs four inputs: workload specification, architecture specification, SAFs specification, and mapping or mapspace constraints. Fig. 6 shows a set of example specifications to show the input semantics, and more detailed syntax can be found at [57].

*Workload specification* describes the shape and statistical density characteristics of the workload tensors (e.g., in Fig. 6, A is 4x4 and has a density of 25% with a uniform distribution). Workload specification also includes the tensor algorithm specification, which is based on the well-known Einsum notation [14, 26] (e.g., the matrix multiplication kernel is specified as  $Z_{m,n} = \sum_k A_{m,k} \times B_{k,n}$ , where the A and B values along the same k dimension are reduced and the m and n dimensions are populated to the output tensor Z). Sparseloop understands any algorithm described with an extended Einsum notation, similar to existing works [18, 36].

*Architecture specification* describes the hardware organization of the architecture (e.g., two levels of storage and four compute units) and the hardware attributes of the component in the architecture (e.g., Backing Storage is 128kB).

*SAFs specification* describes the SAFs applied to the storage or compute levels and the relevant attributes associated with each SAF (e.g., Fig. 6 specifies skipping at Buffer, with A as the leader and B as the follower).

*Mapping* describes an exact schedule for processing the workload on the architecture. It is represented by a set of loops [36]. Each iteration of the for loop represents a time step, and the iterations in a parallel-for loop represent operations happen simultaneously at different spatial instances (e.g., n1s loop shows that different columns of B are simultaneously processed in four Buffers).

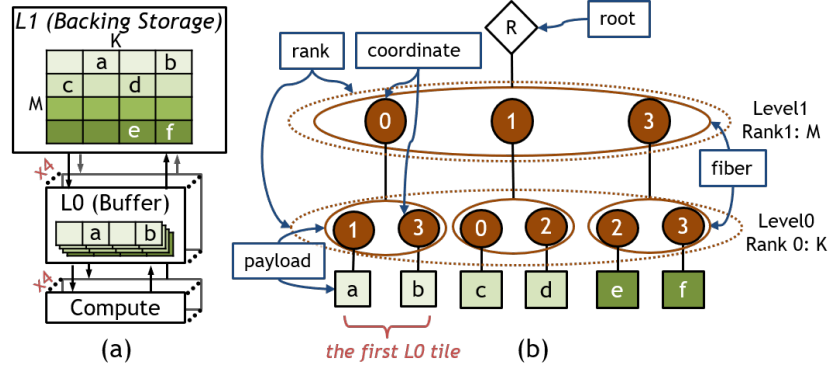


Fig. 7. (a) Example coordinate-space tiling for tensor A based on inputs specified in Fig. 6. The shades represent tiles processed at different time steps. (b) Fiber tree representation of the tensor A. Each level of the tree corresponds to a *rank* of the tensor and contains one or more *fibers* that correspond to the rows or columns of the tensor. The leaves of the tree are the (nonzero) data values of the tensor.

*Mapspace Constraints* describes a set of constraints on allowed schedule (e.g., allowed loop orders). Sparseloop then explores the potential mappings that satisfy the provided partial loops and locates the best one for a specific workload.

## 5.2 Step One: Dataflow Modeling

Dataflow modeling derives the uncompressed data movement and dense compute, which we refer to as the *dense traffic*. Such dense analysis has been studied in several existing works [21, 29, 31, 36, 43]. Since each modeling step in Sparseloop is well-abstracted, various strategies can be plugged into Sparseloop’s modeling process. In our implementation, we adopt Timeloop’s [36] strategy.

Dataflow modeling is performed based on an abstract architecture topology (e.g., Fig. 7a shows the abstract representation of the architecture in Fig. 6), workload tensors’ shapes, and the specified mapping. According to the mapping, each workload tensor is hierarchically partitioned into smaller tiles based on coordinates, with each tile stored in a specific storage level, and this process is referred to as *coordinate-space tiling* [50]. For example, in Fig. 7a, at L1, the tensor A in Fig. 6 is partitioned into four tiles (with different shades of blue) based on the *m1 for loop* in the mapping, each of which is a row of the tensor. Each tile is then sequentially sent to L0. To derive the data movement for each storage level, dataflow modeling analyzes the stationarity of the tiles and the amount of data transferred, both temporally and spatially, between consecutive tiles. The number of computes is derived based on the input tensor algorithm. More detailed description of dense traffic calculations can be found in Timeloop [36].

## 5.3 Step Two: Sparse Modeling

The sparse modeling step is responsible for reflecting the overhead and savings introduced by various SAFs. As shown in Fig. 8, sparse modeling first evaluates the impact of SAFs locally on per-tile traffic with SAF-specific analyzers, i.e., the *Gating/Skipping Analyzer* and the *Format Analyzer*, and then post processes the local traffic with simple scaling to reflect SAFs’ impact on overall traffic.

Such decomposition of local and global traffic analysis allows sparse modeling to reflect SAFs’ impact *on top of* the dense traffic to produce *sparse traffic* for storage and compute units. We now discuss how each module in Fig. 8 interacts with others and the insight behind this design.

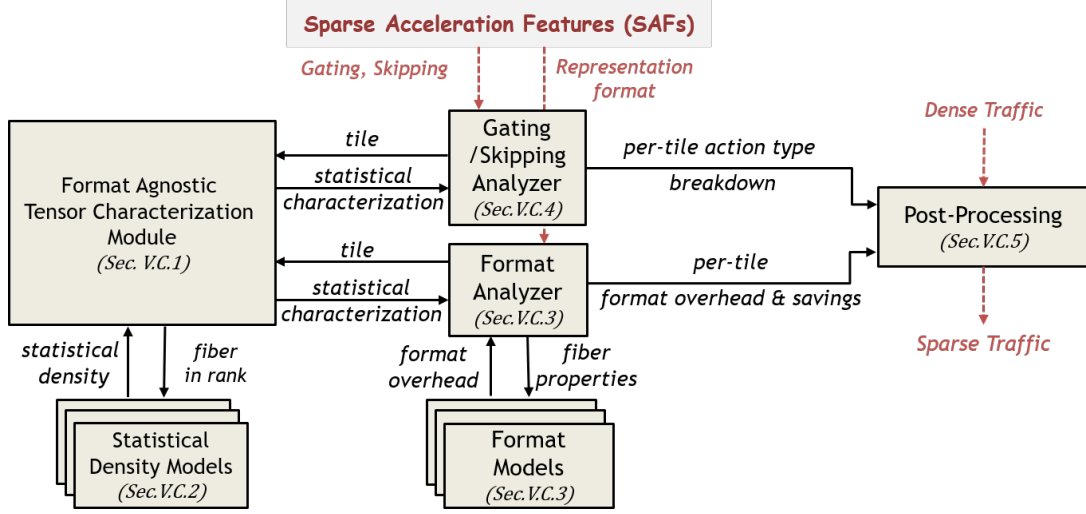


Fig. 8. Various modules in sparse modeling step. Hashed red arrows refer to inputs and outputs of this step. The modules are labeled with their corresponding section numbers.

**5.3.1 Format-Agnostic Tensor Description.** As shown in Fig. 8, to allow tractable complexity and extendability, sparse modeling performs decoupled analysis of SAFs with different analyzers. Describing sparsity characteristics independent of representation format is core to performing such decoupled analysis. We adopt the *fibertree* concept [50] to achieve format-agnostic tensor description. In Fig. 7b, we present the fibertree representation of the sparse tensor A stored in *L1* of Fig. 7a. With the example, we first introduce the key fibertree concepts relevant to Sparseloop.

In fibertree terminology, each dimension of a tensor is called a *rank*<sup>5</sup> and is named. Thus this 2D tensor has 2 ranks, with the rows being named *M* (rank1), and columns being named *K* (rank0). In Fig. 7b, each level of the tree corresponds to a tensor rank in a specific order. Each rank contains one or more *fibers*, representing the rows or columns of the tensor. Each fiber contains a set of coordinates and their associated *payloads*. For intermediate ranks, the payload is a fiber from a lower rank (e.g., coordinate 0 in rank1 has a fiber in rank0 as its payload); for the lowest rank, the payload is a simple value. By omitting the coordinate for all-zero payloads, i.e., empty elements, a fibertree-based description accurately reflects the tensor’s sparsity characteristics (e.g., rank1’s fiber having empty coordinate 2 indicates that the third row is all-zero).

Each fiber in the tree corresponds to a *tile* being processed. For example, in Fig. 7, the first tile processed in *L0* corresponds to the first fiber in *Rank K*. Thus, fibertree-based description enables format-agnostic sparsity-dependent analysis: to analyze the tiles of interest, the analyzers can examine the appropriate fibers to obtain sparsity information independent of the tensor’s representation format.

**5.3.2 Statistical Density Models.** Examining every fiber (thus analyzing the behavior of every tile) is too time-consuming for mapspace and design space exploration. To enable faster analysis, Sparseloop performs statistical characterizations of the fibers in the fibertree. As shown in Fig. 8, Sparseloop can use various statistical density models of the workload tensor to derive statistical density for fibers in each rank (e.g., for the example in Fig. 7b, the fibers in rank0 have a density of 50% with a probability of 0.75 and a density of 0% with a probability of

<sup>5</sup>A rank can also correspond to split or flattened dimensions.

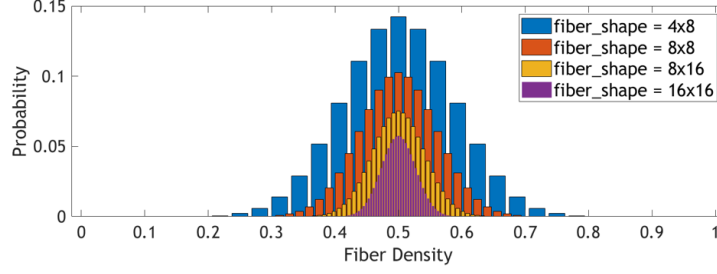


Fig. 9. Fiber density probabilities for fibers with various shapes in a tensor with 50% randomly distributed nonzeros.

| Density Models          | Sparsity Pattern                        | Example Applications                            |
|-------------------------|-----------------------------------------|-------------------------------------------------|
| <b>Fixed structured</b> | Even distribution, Coord. independent   | Structurally pruned DNNs [34]                   |
| <b>Uniform</b>          | Random distribution, Coord. independent | Randomly pruned DNNs [53] & Activation sparsity |
| <b>Banded</b>           | Diagonal distribution, Coord. dependent | SuiteSparse [27], Scientific simulations [3]    |
| <b>Actual Data</b>      | Non-statistical, Coord. dependent       | Graph analytics with special patterns [19]      |

Table 4. Summary of density models supported by Sparseloop. New models can be easily added via Sparseloop’s interface.

0.25). For a given density model, the derived statistical density can differ significantly across fibers with different shapes (*i.e.*, fibers from different ranks in the tree). For example, Fig. 9 shows the distribution of fiber densities in a tensor with uniformly distributed non-zero values. In a uniform distribution, a tile’s shape varies inversely with the deviation in its density.

To estimate the density of fibers with a given shape, a density model either performs *coordinate-independent* modeling (*i.e.*, fibers at different coordinates have similar density distributions) or *coordinate-dependent* modeling (*i.e.*, fiber’s density is a function of its coordinates). Sparseloop supports four popular density models: *fixed-structured*, *uniform*, *banded*, and *actual data*. Table 4 describes their properties and use cases in terms of relevant applications (*e.g.*, randomly pruned DNNs [53] and scientific simulations [3]). The modularized implementation of density models ensures Sparseloop’s extensibility to modeling of emerging workloads with different nonzero value distributions.

**5.3.3 Format Analyzer.** With fibers statistically characterized, the format analyzer is responsible for deriving the representation overhead for the tiles stored in different storage levels. Since different tiles correspond to different fibers, it’s important for the analyzer to identify the tile in each storage and obtain the appropriate statistical characterization of the corresponding fiber from the *format-agnostic tensor characterization module*.

As shown in Fig. 8, for each fiber, the analyzer statistically models the overhead of each rank with the appropriate per-rank *Format Model*. Different formats introduce different amounts of overhead. For example, the *RLE* format model calculates the overhead based on the number of non-empty elements in the fiber,  $Overhead_{RLE} = \# non-empty-elements \times run\_length\_bitwidth$ ; whereas, the bitmask (*B*) format model produces the same overhead regardless of fiber density,  $Overhead_B = total\ \# elements \times 1$ . The statistical format overhead allows Sparseloop to derive important analytical estimations, *e.g.*, the average and worst-case overhead. Sparseloop supports five per-rank format models: *B*, *CP*, *UOP*, *RLE*, and *Uncompressed B*, and thus supports any representation format that can be described with these models. The framework can be easily extended to support other formats.

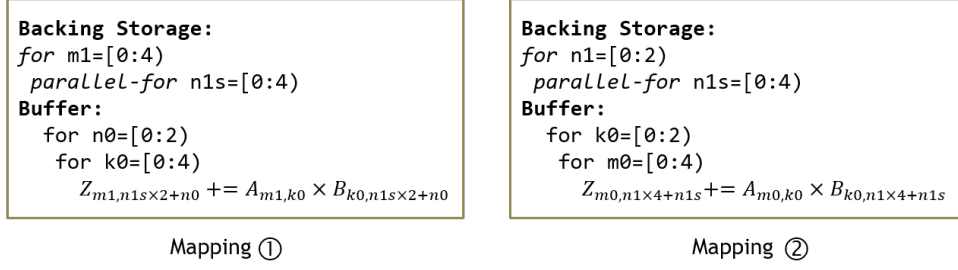


Fig. 10. Example mappings that lead to intersections with different impact.

**5.3.4 Gating/Skipping Analyzer.** The *Gating/Skipping Analyzer* evaluates the amount of eliminated IneffOps introduced by each gating/skipping SAF. Since gating/skipping focuses on improving efficiency for each tile being transferred and/or each compute being performed, regardless of the total number of operations, the analyzer evaluates the impact of SAFs locally on per-tile traffic and breaks down the original per-tile dense traffic into three fine-grained action types: i) actually happened, ii) are skipped, and iii) are gated.

As discussed in Sec. 3, gating/skipping is based on various intersections, which eliminate IneffOps by locating the empty tiles, *i.e.*, tiles with all zeros. In a leader-follower intersection, when the leader tile is empty, the IneffOps associated with the follower are eliminated. Whereas in a double-sided intersection, any tile being empty leads to eliminations of IneffOps associated with the other tile. Since a double-sided intersection can be modeled as a pair of leader-follower intersections ( $B \leftrightarrow A = B \leftarrow A + A \leftarrow B$ ), we focus on discussing the modeling of SAFs based on leader-follower intersections.

The key to modeling the amount of eliminated IneffOps introduced by a SAF based on leader-follower intersection is to correctly identify the associated leader and follower tiles, and thus the fibers representing the tiles in the fibertree. Since different fibers can have significantly different probability of being empty, the same SAF can lead to different impacts. We observe that the leader and follower tiles of a specific intersection can be determined based on the data reuse defined in the mapping. For example, Fig. 10 shows two mappings that lead to different intersection behaviors for *Skip*  $B \leftarrow A$  at *Buffer*. In *Mapping ①*, since the innermost  $k0$  loop iterates through different pairs of  $A$  and  $B$  values, a specific  $B_{k,n}$  is only used to compute with a single  $A_{m,k}$ . Thus, the leader tile is a single  $A$  value and the follower tile is a single  $B$  value, *i.e.*, if  $A_{m,k}$  is zero, the access to  $B_{k,n}$  will be eliminated. Whereas in *Mapping ②*, since the innermost  $m0$  loop only iterates through different  $A$  values, a specific  $B_{k,n}$  is reused across  $A_{0:3,k}$  (*i.e.*, a column of  $A$ ). Thus, the leader tile is  $A_{0:3,k}$  and the follower tile is a single  $B$  value  $B_{k,n}$ , *i.e.*, if the entire column of  $A$  is empty, the access to  $B_{k,n}$  will be eliminated. Since it is less likely for the entire column of  $A$  to be empty, under *Mapping ②*, *Skip*  $B \leftarrow A$  eliminates fewer IneffOps (*e.g.*, columns of  $A$  are never empty in Fig. 6).

Based on the mapping and the statistical fibertree characterization, the analyzer defines the behaviors of each gating or skipping SAF, and derives a breakdown of the original dense traffic for each tile into fine-grained action types (*e.g.*, for each  $B$  tile transferred from *Buffer* to *Compute*, there are 50% skipped reads, 50% actual reads, 0% gated reads). Furthermore, when a gating/skipping SAF is applied to upper storage levels in the architecture, the analyzer propagates the savings introduced to lower levels (*e.g.*, for the architecture in Fig. 6, skipping at *Backing Storage* reduces operations happened at both *Buffer* and *Compute*).

**5.3.5 Traffic Post-processing.** As shown in Fig. 8, after the analyzers evaluate the impact of their respective SAFs based on per-tile traffic, sparse modeling performs post-processing to first reflect the interactions between



the SAFs (e.g., how much format overhead is skipped due to a skipping SAF) and then scale the per-tile breakdowns based on the number of tiles transferred to derive the final sparse traffic.

#### 5.4 Step Three: Micro-architectural Modeling

Micro-architecture modeling first evaluates the validity of the provided mapping. A mapping is valid only if the largest tiles, which are derived based on statistical tile densities and format overheads, meet the capacity requirement of their corresponding storage levels. If the mapping is valid, micro-architecture modeling evaluates the impact of micro-architecture on generated sparse traffic. The analysis focuses on capturing general micro-architectural characteristics, e.g., segmented block accesses for storage levels, instead of the design-specific micro-architectural analysis, e.g., impact of an exact routing protocol.

Micro-architectural modeling then evaluates the processing speed and energy consumption. For processing speed, cycles are spent for *actual* and *gated* storage accesses and computes. The model considers available bandwidth at each level in the architecture to account for bandwidth throttling. For energy consumption, we use an energy estimation back end (e.g., Accelergy [54]) to evaluate the cost of each fine-grained action, which is combined with its corresponding sparse traffic to derive accurate energy consumption.

### 6 EVALUATIONS

We first introduce our experimental methodology and then demonstrate that Sparseloop is fast and accurate.

#### 6.1 Methodology

Sparseloop is implemented in C++ on top of Timeloop[36], an analytical modeling framework for *dense tensor accelerators*. Sparseloop reuses Timeloop’s dataflow analysis, adds the new sparse modeling step, and improves Timeloop’s micro-architectural analysis to account for the impact of various fine-grained actions introduced by the SAFs. *As a result, Sparseloop allows modeling of both dense and sparse tensor accelerators in one unified infrastructure.* We use Accelergy [54, 55] as the energy estimation back end. For DNN workloads, Sparseloop performs per-layer evaluations *with the appropriate dataflow and SAFs*, and aggregates the results to derive the energy/latency for the full network. This methodology is consistent with state-of-the-art tensor accelerator modeling frameworks [21, 29, 31, 36]. Experimental results in the next sections are all evaluated on an Intel Xeon Gold 6252 CPU.

#### 6.2 Simulation Speed

Fast modeling speed allows designers to quickly explore each design’s large mapspace as well as various designs. We evaluate Sparseloop’s modeling speed with the metric *computes simulated per host cycle* (CPHC), which refers to the number of accelerator computes simulated for each cycle in the host machine that runs the modeling framework. CPHC carries similar information as MIPS (million instructions per second), a popular metric for evaluating simulators for conventional processors.

Detailed cycle-level simulators often have CPHCs that are lower than 1. For example, STONNE [30] has CHPCs that are less than 0.5 when running popular DNN layers with various architecture configurations, e.g., number of rows and columns in the compute array. The main reasons include: i) instead of statistical analysis, cycle-level simulators iterate through actual data to perform all computations, which take significant time for large workloads with millions of computations such as DNNs; ii) detailed control logic needs to be simulated for every cycle and all the components (e.g., memory interface protocols, exact intersection checks).

Sparseloop achieves much higher CPHCs with its analytical modeling approach since Sparseloop avoids performing analysis on all computations by performing statistical analysis on transient and steady state design behaviors only; and does not simulate detailed cycle-level control logic. Table 5 shows Sparseloop’s CPHCs for example DNN accelerators [8, 9, 37] running representative workloads [13, 17, 28, 45]. The CPHCs are dependent

| Accelerator Designs | Workloads |           |       |         |
|---------------------|-----------|-----------|-------|---------|
|                     | ResNet50  | BERT-base | VGG16 | AlexNet |
| Eyeriss             | 5.2k      | 13.3k     | 53.8k | 21.4k   |
| Eyeriss V2 PE       | 2.7k      | 12.5k     | 20.4k | 13.2k   |
| SCNN                | 1.1k      | 4.3k      | 3.7k  | 5.2k    |

Table 5. *Computes simulated per host cycle (CPHC)* for designs modeled by Sparseloop. Compared to cycle-level tensor accelerator simulator STONNE [30], which has less than 0.5 CPHC, Sparseloop is over 2000× faster.

| Accelerator Design | Baseline Model                                              |                                                 |                  |                                    | Average Accuracy | Major Sources of Error                                                                   |
|--------------------|-------------------------------------------------------------|-------------------------------------------------|------------------|------------------------------------|------------------|------------------------------------------------------------------------------------------|
|                    | Source                                                      | Type                                            | Sparsity Pattern | Output                             |                  |                                                                                          |
| SCNN               | Simulators obtained from authors [9, 37]                    | Design-specific                                 | Statistical      | Runtime activities                 | 99.9%            | None                                                                                     |
| Eyeriss V2 PE      |                                                             | Analytical                                      | Actual           | Processing latency                 | >98%             | Statistical approximations                                                               |
| Eyeriss            | Results directly from paper or technical report [8, 34, 53] | Real hardware                                   |                  | Compression rate<br>Energy savings | >95%             | (1) Statistical approximations<br>(2) Approximated component energy characterizations    |
| DSTC               |                                                             | Cycle-level simulator validated on silicon [25] |                  | Processing latency                 | 92.4%            | (1) Statistical approximations;<br>(2) Optimistic modeling of microarchitectural details |
| STC                |                                                             | Real hardware                                   |                  | Processing latency                 | 100%             | None<br>(structured sparsity introduces deterministic behaviors)                         |

Table 6. High-level summary of performed validations based on available data from existing work. Overall, Sparseloop achieves 0.1% to 8% average error across different designs. More details in Sections 6.3.1, 6.3.2, 6.3.3, 6.3.4 and 6.3.5.

on accelerator architecture characteristics (e.g., SAFs complexity, number of levels, etc.), employed dataflow, and DNN workload characteristics (e.g., sparsity, tensor shapes, number of layers, etc.). For example, compared to Eyeriss V2 and SCNN, Eyeriss’ less powerful SAF support (more details in Table 3) always introduces lower SAF modeling complexity and more simulated computes, leading to a higher CPHC. Overall, Sparseloop is over 2000× faster compared to STONNE [30].

### 6.3 Validation

High modeling accuracy, in terms of both absolute values and relative trends, allows designers to correctly analyze design trade-offs at an early stage. To demonstrate Sparseloop’s accuracy, we validate on five well-known accelerator designs: SCNN [37], Eyeriss [8], Eyeriss V2 [9], and dual-side sparse tensor core (DSTC) [53], and Sparse Tensor Core (STC) [34]. **Overall, Sparseloop maintains relative trends and achieves 0.1% to 8% average error.** Based on available information from existing work, validations are performed on baseline models that capture increasing amount of design details: from analytical models based on statistical sparsity patterns to cycle-level models/real hardware designs based on actual sparsity patterns. At a high-level, common sources of error include: 1) statistical approximation of actual data 2) approximated component characteristics 3) approximated impact of design-specific micro-architectural implementations. Table 6 summarizes the validations.

In the next sections, we present more detailed validation discussions. In order to validate our work against prior works, we need to use the workloads reported in those works, despite the reported workloads being old (though popular at the time of the work’s publication) or different across designs. This is mainly due to the fact that other workloads are either not available in the reported results or not directly supported by the available simulators.

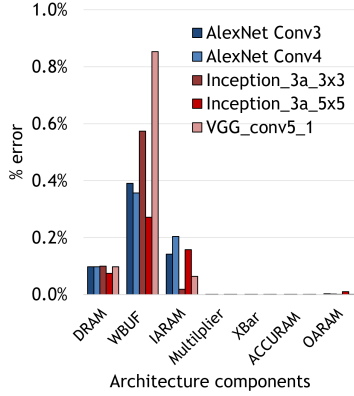


Fig. 11. Runtime activity (i.e., storage access and compute count) validation for SCNN [37]. Achieves less than 1% error for all storage and compute components in the architecture.

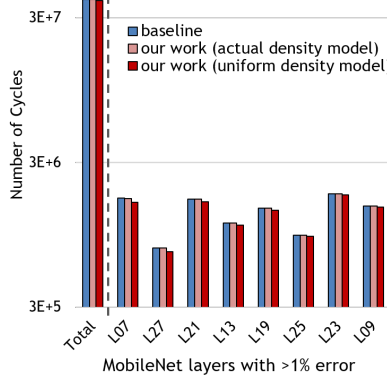


Fig. 12. Processing latency validation for Eyeriss V2 processing element [9] running MobileNet [20]. For ease of presentation, we only show total cycle counts and layers with more than 1% error.

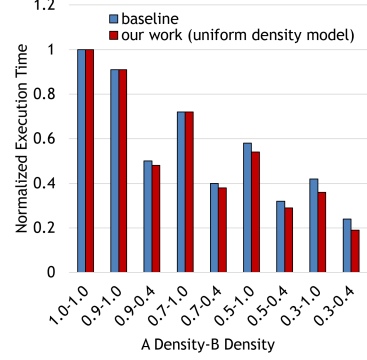


Fig. 13. Processing latency of dual-side sparse tensor core [53] running matrix multiplication workloads with various operand tensor densities, normalized to dense processing latency. Average error is 7.6%.

**6.3.1 SCNN.** We first validate Sparseloop on SCNN [37] with a customized simulator that was used in the paper: it performs analytical modeling based on *statistically* characterized data. SCNN baseline assumes uniform distribution and captures the runtime activities of the components in the architecture (e.g., the number of reads and writes to various storage levels). Fig. 11 shows the error rate of the runtime activities for each component in the architecture. Sparseloop, running with a uniform density model, is able to capture all runtime activities accurately with less than 1% error for all components in the architecture.

**6.3.2 Eyeriss V2.** Since Eyeriss V2’s SAFs are implemented in its processing element (PE), we focus on validating the PE based on a baseline model that performs *actual sparsity pattern* based analytical modeling. To quantitatively demonstrate the sources of error, we validate Sparseloop with both an actual-data density model and a uniform density model.

Fig. 12 shows the validation on the number of cycle counts. In terms of total cycle counts for processing the entire MobileNet [20], Sparseloop achieves more than 99% accuracy and is able to capture the relative trends across different layers with both density models. However, for layers with both sparse operands compressed, modeling based on a uniform density model results in up to 7% error for layer27. Fig. 12 shows the layers with more than 1% error. The errors are mainly attributed to the statistical approximation of the expected nonempty intersection ratio between two sparse tensors, as the exact nonempty ratio deviates from case to case, e.g., when both operands have identical nonzero value locations, the intersection nonempty ratio is equal to the tensor densities. With an actual-data density model, Sparseloop accounts for the exact intersections, and thus accurately captures the cycle counts (at the cost of a slower modeling speed).

**6.3.3 Dual-Side Sparse Tensor Core.** For DSTC, the baselines are also obtained directly from the papers whose reported results are based on a cycle-level simulator that is validated on real hardware [25]. We validate on the normalized processing latency running matrix multiplication workloads with various operand tensor density degrees, as shown in Fig. 13. We modeled the tensors with a uniform density model, captured the performance trends across density degrees, and obtained an average error of less than 8%. In addition to errors introduced by deviations from the expected nonempty intersection ratio, Sparseloop also performs optimistic modeling of

|                    | Conv1 | Conv2 | Conv3 | Conv4 | Conv5 |
|--------------------|-------|-------|-------|-------|-------|
| <b>Eyeriss</b> [8] | 1.2   | 1.4   | 1.7   | 1.8   | 1.9   |
| <b>Sparseloop</b>  | 1.2   | 1.4   | 1.7   | 1.9   | 1.9   |

Table 7. Eyeriss[8] DRAM compression rate validation.

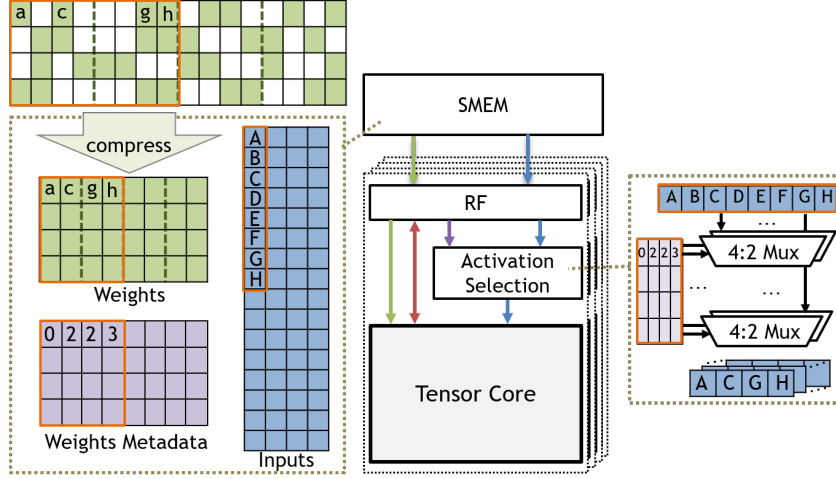


Fig. 14. Modeled sparse tensor core architecture (including the *SMEM* in streaming processor for a more holistic view) and example processing of a 2:4 structured workload.

micro-architectural details. More specifically, Sparseloop assumes no storage bank conflicts but DSTC’s baseline results contain bank conflicts when operand tensors are relatively sparse (e.g., 30% density), thus introducing higher processing latency.

**6.3.4 Eyeriss.** We validate on Eyeriss [8] with baselines obtained from the paper and based on taped-out silicon. We first validate DRAM compression rates for AlexNet [28], as shown in Table 7. Overall, we achieve 1% error on average and the discrepancy could be due to imperfect compression with the actual data. We also validate on the PE array energy reduction ratio due to on-chip gating. Eyeriss claims that the energy savings of the processing elements can achieve 45%. Our results show a max energy efficiency improvement of 43%. The discrepancy could be due to not modeled PE components with unknown energy characteristics.

**6.3.5 Sparse Tensor Core.** Finally, we validate the Ampere GPU’s sparse tensor core accelerator (STC) based on publicly available architecture descriptions [10, 34, 49]. STC focuses on accelerating structured sparse workloads with a 2:4 sparsity structure, which demands at most two nonzero values in every block of four values. Fig. 14 shows the high-level STC architecture and an example processing flow of a 2:4 structured sparse matrix multiplication workload (algorithm defined in Fig. 6). We will discuss more details about STC in Section 7.1.

To validate STC, we use the fixed structured density model parameterized with the 2:4 structure along each channel to model the structured sparse weight tensor. Existing work reports that STC achieves 2× speedup compared to dense processing [10, 34, 49]. Because of the fully defined behaviors with the structured sparsity, Sparseloop also produces an exact 2× speedup (STC design in Fig. 15), achieving 100% accuracy.

## 7 CASE STUDIES

In this section, we demonstrate Sparseloop’s flexibility with two case studies.

### 7.1 Investigating Next Generation Sparse Tensor Core

In recent years, various techniques have been proposed to add sparsity support to tensor core (TC). In this case study, we use Sparseloop to first compare two variations: the commercialized NVIDIA STC [34] and a research-based proposal DSTC [53]. Based on the comparison, we then discuss the potential opportunities for next-generation STC, and showcase an example design flow that uses Sparseloop to identify current STC design’s limitations and explore various solutions to such limitations to unlock more potential.

**7.1.1 DSTC vs. NVIDIA STC.** We perform an apples-to-apples comparison of the two designs. Since both designs are TC-based, both architectures contain the *SMEM-RF-Compute* hierarchy as shown in Fig. 14, and are controlled on allocated hardware resources, including compute, storage capacity, and memory bandwidth. To model realistic systems, we only provision a subset of a real GPU’s *SMEM* bandwidth to the accelerators, since other processes running on the GPU share the same *SMEM* storage. At a high-level, DSTC employs complex sparsity support and a special outer product dataflow to exploit **arbitrary sparsity in both operands** to perform compression and skipping. In contrast, STC ignores input sparsity and uses low-overhead sparsity support to compress and perform skipping **on weights with 2:4 structured sparsity only**.

Fig. 15 compares the cycles spent and energy consumed by DSTC and STC running ResNet50 [17] pruned to various sparsity degrees. ResNet50 contains sparse weights (if pruned) and sparse inputs. Compared to STC, DSTC’s dataflow for supporting arbitrary sparsity incurs a significant amount of data movement. As a result, when processing denser workloads (e.g., unpruned ResNet50 in this example or BERT-like networks with dense input activations), even if DSTC is able to always introduce lower cycles counts, the savings brought by SAFs cannot compensate for the additional energy spent and thus the overall hardware efficiency is low. However, STC provides very limited support for different workloads. Furthermore, for sparser workloads, e.g., 25% dense ResNet50 in Fig. 15(a), despite DSTC’s overhead, it’s able to achieve a much higher overall efficiency because of the speedup introduced by a significant amount of skipping.

**Opportunities for STC: only supporting 2:4 sparsity in the current STC design leads to missed opportunities, as many modern DNNs can be pruned to >50% sparsity (structured [67] or unstructured [16]) while maintaining reasonable accuracy. Thus, one possible feature for a next generation STC to have is to efficiently exploit the savings brought by more sparsity degrees but still keep the sparsity structured to reduce SAF overhead.**

**7.1.2 Naive STC Extension To Support More Ratios.** In order to extend the existing STC to support more sparsity degrees, we first introduce the existing high-level processing of STC running matrix multiplication workloads (algorithm defined in Fig. 6) with the default 2:4 structured sparsity. In the case of a DNN, tensor A in Fig. 6 corresponds to the structured sparse weights in Fig. 14.

As shown in Fig. 14, the weight tensor is compressed with an offset-based coordinate payload format, where each nonzero carries an offset coordinate to indicate its position in the block of four values, e.g., the nonzero weight  $g$  is the third element in its block, and thus carries a metadata of 2. This format matches our *CP* format in earlier sections. The compressed weight tensor and the uncompressed input tensor are stored in *SMEM*. For each iteration of processing, the weights, weight metadata, and dense inputs are fetched out. However, since inputs are uncompressed, as shown in Fig. 14, a *tile with four weights corresponds to a tile with eight inputs*. Thus, to ensure correctness, a 4:2 selection needs to be performed on the inputs for each block of four weights. Since only nonzero weights need to be processed, the 2:4 processing is  $2\times$  faster than dense processing.

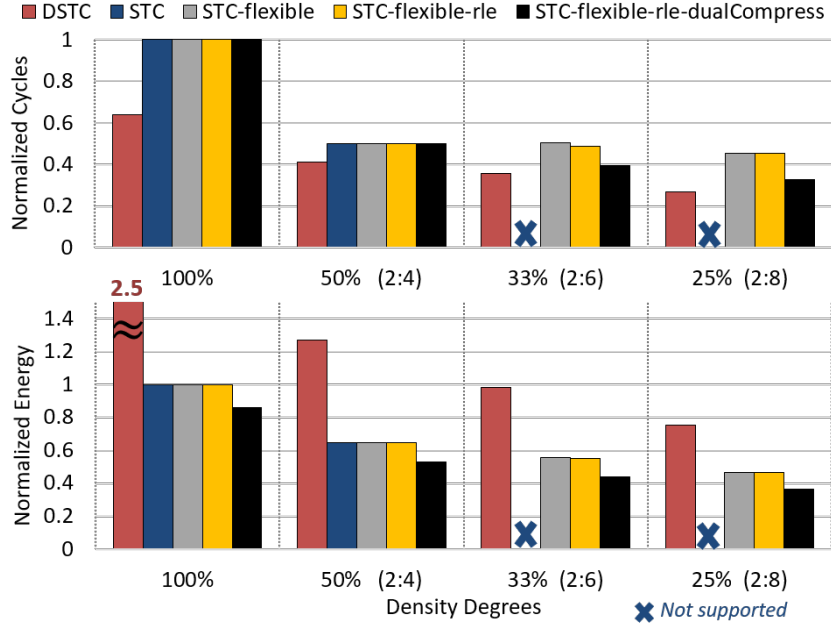


Fig. 15. Sparseloop’s analysis on the normalized total cycles spent and energy-delay product for various designs of tensor core accelerator running representative ResNet50 [17] layers pruned to various sparsity degrees. The accelerators are controlled to have similar amount of hardware resources.

Thus, naively supporting more sparsity degrees in STC simply involves extending the above discussed sparsity support with input activation selection logic for more ratios, *e.g.*, 2:6 and 2:8. We name this naive extension as *STC-flexible*. As shown in Fig. 15, Sparseloop’s modeling indicates that *STC-flexible* does support and introduce extra energy reductions for lower density workloads. However, no desirable speedup is obtained with the higher sparsity, *e.g.*, theoretically, 2:6 structured sparsity should introduce 3× speedup. **In fact, surprisingly, the baseline processing barely brings any additional speedup with the naive extension for 2:6 and 2:8 workloads.**

**7.1.3 Identify Design Limitations.** *STC-flexible*’s approach does not improve performance due to *SMEM* bandwidth limitation. Fig. 16 shows Sparseloop’s analysis on the required bandwidth for processing workloads with various sparsity ratios. To ensure full utilization of the tensor core, the same number of nonzero weights needs to be processed spatially regardless of the workload sparsity, *i.e.*, we always need 1× weights as shown in Fig. 16. As discussed above, STC stores inputs in uncompressed format. Thus, the sparser the weight tensor, the more inputs need to be fetched in a cycle, *e.g.*, in Fig. 16, 4× inputs need to be fetched for workloads with 2:8 sparse weights. In addition to the bandwidth pressure imposed by inputs, the metadata also needs to be described with more bits as the block size gets larger. The amount of additional metadata overhead is dependent on the chosen representation format, *e.g.*, run length encoding (RLE) requires fewer bits than offset-based CP for 2:6 sparse workloads. **As a result, STC is bottlenecked by the limited bandwidth, which is provisioned for 2:4 structured sparsity, and thus cannot obtain the theoretical speedup for sparser workloads.**



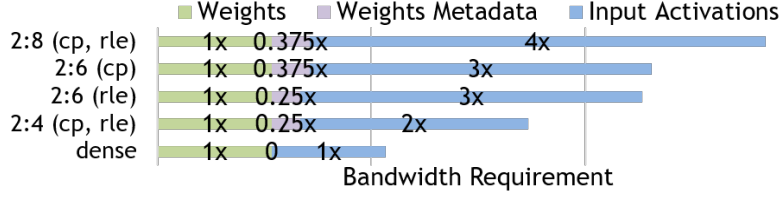


Fig. 16. Sparseloop’s analysis on bandwidth requirements for getting ideal speedup for various operands and associated metadata (if any).

**7.1.4 Explore Solutions to Overcome Limitations.** With Sparseloop, we can perform early design stage exploration on potential solutions. Without loss of generality and for the ease of presentation, we discuss two example directions with low-hanging fruit: 1) improve representation format support to reduce metadata overhead; 2) introduce additional compression SAFs for inputs.

First, we evaluate if a different representation (compression) format can alleviate the overhead introduced by metadata, especially for 2:6 structured sparsity. Thus, as shown in Fig. 15, we enabled *RLE* support for *STC-flexible* to form *STC-flexible-rle*. At a high-level, compared to the STC’s original *CP* support, *RLE* support does provide similar or better processing speed. However, since the majority of the overhead comes from transferring actual data, the benefits are too insignificant to bring *STC-flexible-rle* over DSTC.

We then target the more important bottleneck: the uncompressed input data traffic. To solve the problem, we added bitmask-based compression to input such that both operands are compressed to form *STC-flexible-rle-dualCompress* design in Fig. 15. To keep the compute easily synced, we did not add input-based skipping. As a result, all of the obtained speedups come from bandwidth requirement reduction. As shown in Fig. 15, *STC-flexible-rle-dualCompress* can actually introduce similar speed even if it cannot exploit input sparsity for skipping. This is because even if DSTC exploits both operands for speedup, its dataflow has more frequent streaming of operands, introducing additional pressure to *SMEM* bandwidth as well. **Thus, with this example, we have demonstrated that exploiting more sparsity does not guarantee more speedup, and it is very important to make sure the dataflow and SAF overhead is reasonable.**

Overall, as shown in Fig. 15, we derived *STC-flexible-rle-dualCompress* that, compared to DSTC, always introduces lower energy consumption and has similar processing speed most of the time for the studied sparsity degrees.

## 7.2 Co-design of Dataflow, SAFs and Sparsity

Looking beyond the deep learning workloads and tensor core accelerators discussed in the previous case study, this section demonstrates how Sparseloop can model workloads with more diverse sparsity degrees and accelerator designs that employ various dataflows and SAFs. With a set of small-scale experiments, we show various broad insights for designing sparse tensor accelerators: (1) the best design for one application domain might not be the best for another; (2) combining more energy or latency saving features together does not always make the design more efficient. Thus, careful co-design of dataflow, SAFs and sparsity is necessary for achieving desired latency/energy savings.

**7.2.1 Design Choices. Workloads:** We use matrix multiplication with sparse input tensors (spMspM) of various density degrees as example workloads. spMspM, represented as  $Z_{m,n} = \sum_k A_{m,k} \times B_{k,n}$  as an Einsum, is an important kernel in many popular applications, such as scientific simulations, graph algorithms and DNNs, each of which can have different tensor density degrees.

(a)

| Dataflow Choices | Tensor Reuse      |               |                   |
|------------------|-------------------|---------------|-------------------|
|                  | A                 | B             | Z                 |
| <b>ReuseABZ</b>  | Innermost storage | Shared buffer | Innermost storage |
| <b>ReuseAZ</b>   | Innermost storage | None          | Innermost storage |

(b)

| SAFs Choices            | Operand Intersection      |                           |
|-------------------------|---------------------------|---------------------------|
|                         | Off-chip                  | On-chip                   |
| <b>InnermostSkip</b>    | None                      | $SkipB \leftrightarrow A$ |
| <b>HierarchicalSkip</b> | $SkipB \leftrightarrow A$ | $SkipB \leftrightarrow A$ |

Table 8. Choices for different design aspects: (a) dataflows (b) SAFs (representation formats and other minor SAFs are identical and thus are not shown for simplicity)

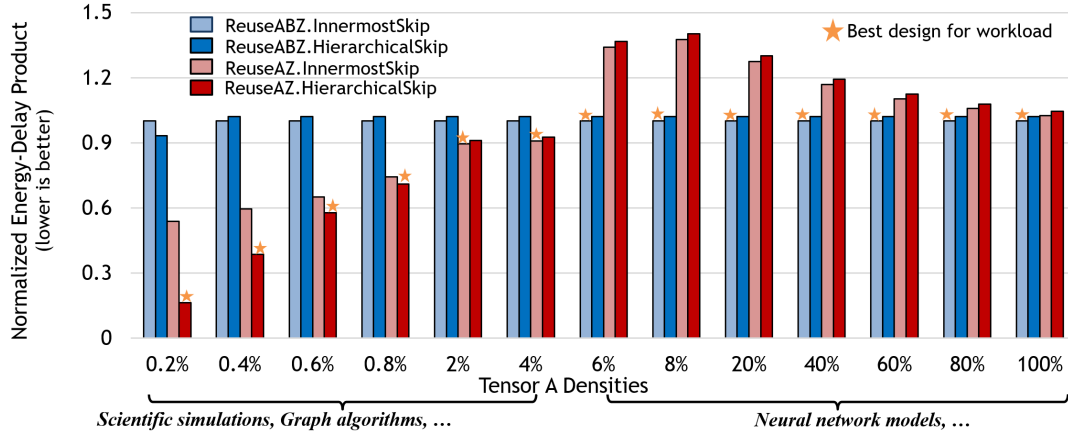


Fig. 17. Normalized energy-delay product of different combinations of dataflow-SAFs running matrix multiplications with various density degrees, which are labeled with relevant example workloads. Sparseloop shows (1) dataflow and SAFs should be co-designed to ensure potential savings; (2) the correct combination needs to be chosen for different applications to realize the potential savings.

**Dataflows:** Given a hardware budget of 256 compute units and 128KB on-chip storage, we consider two choices shown in Table 8(a): (1) *ReuseABZ* that reuses all tensors on-chip; (2) *ReuseAZ* that doesn't have on-chip reuse for B.

**SAFs:** As shown in Table 8(b), we consider two sets of SAFs choices: (1) *InnermostSkip* that performs  $SkipB \leftrightarrow A$  at the innermost on-chip storage (2) *HierarchicalSkip* that hierarchically performs  $SkipB \leftrightarrow A$  at DRAM and innermost storage to reduce both off-chip and on-chip data movement.

**7.2.2 Interactions Among Design Choices.** Fig. 17 compares the energy-delay-product (EDP) of different dataflow-SAF combinations running spMspM with various A tensor density degrees. At each density degree, the EDPs are normalized to *ReuseABZ.InnermostSkip*'s EDP.

We first make the observation that the best design for one application domain might not be the best for another. For example, while *ReuseABZ.InnermostSkip* is the best design for NN workloads (i.e., A density >6%), for sparser workloads, such as scientific simulations or graph algorithm, this design is sub-optimal due to its large off-chip bandwidth requirement. On the other hand, *ReuseAZ.HierarchicalSkip* performs the best with hyper-sparse workloads since this design performs early off-chip traffic eliminations, but it fails to reduce EDP with NN workloads due to its inability to perform effective off-chip intersections on denser operands and its lack of on-chip B reuse. Thus, a design’s dataflow-SAFs combinations need to be chosen based on target application’s sparsity characteristics to realize potential savings.

We also show that combining more energy or latency saving features together does not always make the design more efficient. For example, *ReuseABZ.HierarchicalSkip* combines a dataflow that reuses all tensors with SAFs that skip both off-chip and on-chip traffic to form a design with the most number of latency/energy savings features. However, as shown in Fig. 17, *ReuseABZ.HierarchicalSkip* is never the best design in terms of EDP. This is because the *ReuseABZ* dataflow prevents the off-chip skipping SAF from eliminating B’s off-chip data movement. More specifically, since *ReuseABZ* reuses each on-chip B tile for multiple A tiles, B tile transfers can be eliminated by the off-chip skipping SAF only when all values in its corresponding A tiles are zeros, which rarely happens. **Thus, dataflow and SAFs need to be carefully co-designed to ensure there exist opportunities for reasonable savings.**

## 8 RELATED WORK

There is ample prior work in modeling frameworks for tensor accelerator designs. These models can be classified into two classes: *cycle-level models* and *analytical models*.

**Cycle-level models** evaluate the detailed cycle-level behaviors of potential designs. Many of them assume a specific target platform, such as ASIC [52] or FPGA [32, 40, 63] and perform register-transfer-level (RTL) analysis, which includes low-level hardware details (e.g., pipeline stages). There are also platform-independent models, e.g., STONNE[30], which perform cycle-level architectural analysis without RTL implementations. While these cycle-level models are very accurate, they hinder the exploration among the vast number of dataflows due to their long simulation time [6, 23, 36]. Furthermore, cycle-level models are often not well parameterized in terms of architecture topology, employed SAFs, dataflow, etc. These assumptions adversely limit the explorable designs.

**Analytical models** [21, 24, 29, 36, 43, 55, 59, 60, 66] perform higher level analytical evaluations without considering per-cycle processing details of the design. Since these models work on abstracted hardware models, they are usually well parameterized and modularized to support a wider range of architecture designs. However, to the authors’ knowledge, they either do not recognize the sparse workloads or SAFs at all [21, 24, 29, 36, 43, 55, 66], or only target design-specific SAFs [59, 60]. For example, Procrustes[59] supports modeling of B format for one operand only. That is, no prior work aims to flexibly model general sparse tensor accelerators with various SAFs applied. Since at each architecture level, different SAFs can introduce different amounts of savings and overhead, the lack of trade-off analysis for SAFs prevents designers from using such analytical models for design space exploration.

## 9 CONCLUSION

Sparse tensor accelerators are important for efficiently processing many popular workloads. However, the lack of a unified description language and a modeling infrastructure to enable exploration of various designs impedes further advances in this domain. This paper proposes a systematic classification of sparsity-aware acceleration techniques into three high-level sparse acceleration features (SAFs): representation format, gating, and skipping. Exploiting this classification, we develop Sparseloop, an analytical modeling framework for sparse tensor accelerators. We further observe that the analyses of dataflow, SAFs, and micro-architecture are orthogonal

to each other. Based on the orthogonality, we design Sparseloop’s internal analysis as three decoupled steps to keep its modeling complexity tractable. To balance modeling accuracy and simulation speed, Sparseloop uses statistical characterizations of tensors.

Sparseloop is over  $2000\times$  faster than cycle-level simulations, and models well-known sparse tensor accelerators with accurate relative trends and 0.1% to 8% average error. With case studies, we demonstrate that Sparseloop can be used in accelerator design flows to help designers to compare and explore various designs, identify performance bottlenecks (*e.g.*, memory bandwidth), and reveal broad design insights (*e.g.*, co-design of sparsity, SAF and dataflow).

## ACKNOWLEDGMENTS

We thank Haoquan Zhang for discussions on statistical analysis of tensor density. We would also like to thank the anonymous reviewers for their constructive feedback.

Part of the research was done during Yannan Nellie Wu’s internship at NVIDIA Research. This research was funded in part by the U.S. Government. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the U.S. Government. This research was also funded in part by DARPA contract HR0011-18-3-0007, NSF PPOSS 2029016 and Ericsson.

## A ARTIFACT APPENDIX

### A.1 Abstract

In this artifact, we provide the source code of Sparseloop, its energy estimation backend based on Accelergy [54], and input specifications to key experimental results presented in the paper. To allow easy reproduction, we provide a docker environment with all necessary dependencies, automated scripts, and a Jupyter notebook that includes detailed instructions on running the evaluations. The artifact can be executed with any X86-64 machine with docker support and more than 10GB of disk space.

### A.2 Artifact check-list (meta-information)

- **Algorithm:** Analytical modeling of sparse tensor accelerator performance (energy and cycles).
- **Program:** C++, python.
- **Run-time environment:** Dockerfile.
- **Hardware:** Any X86-64 machine.
- **Output:** Plots or tables generated from scripts.
- **Experiments:** Analytical modeling of various sparse tensor accelerators running various workloads.
- **How much disk space required (approximately)?:** 10GB
- **How much time is needed to prepare workflow (approximately)?:** Less than 30min if directly pulling docker image; less than 2 hours if building docker from the source.
- **How much time is needed to complete experiments (approximately)?:** Less than 1 hour to finish running all experiments in the provided default mode.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT
- **Archived (provide DOI)?:** Yes, DOI 10.5281/zenodo.7027215

### A.3 Description

**A.3.1 How to access.** The artifact is hosted both on github (<https://github.com/Accelergy-Project/micro22-sparseloop-artifact>) and on an archival repository with DOI 10.5281/zenodo.7027215 (<https://doi.org/10.5281/zenodo.7027215>).

### A.4 Installation

Since we provide a docker, the installation process mainly involves obtaining the docker image that contains the dependencies, the compiled Sparseloop, and the energy estimation backend. Please follow the provided instructions (<https://github.com/Accelergy-Project/micro22-sparseloop-artifact/blob/main/README.md>) to obtain and start the docker.

### A.5 Evaluation and expected results

We provide a jupyter notebook in *workspace/2022.micro.artifact/notebook/artifact\_evaluations.ipynb* to guide through the evaluations. Please navigate to the notebook in your docker Jupyter notebook file structure GUI.

Each cell in the notebook provides the background, instructions, and commands to run each evaluation with provided scripts. The evaluations include the following key results from the paper:

- Comparison of performance and energy for accelerators supporting different representation formats (Fig. 1).
- Validations on various sparse tensor accelerators (Fig. 12, Table 7, Fig. 13, and the STC design in Fig. 15.)
- Example design flow using Sparseloop to perform apples-to-apples comparison, identify design limitations, and explore various solutions to the limitation (Fig. 15).

The output of each evaluation will either produce a figure or the content of a table. The easiest way to check validity is to compare the generated figure/table with the ones in the paper. However, raw results can also be accessed in the *workspace/evaluation\_setups* folder. Please note that we had to use energy estimation data based on public technology node instead of our proprietary technology node, so the exact data might not match for certain evaluation(s). We explicitly point out such cases in the notebook.

#### A.6 Experiment customization

The input specifications in the *workspace/evaluation\_setups* folder can be updated to specify different hardware setups (e.g., different buffer sizes). Moreover, we also provide options in the scripts to enable map space search using Sparseloop (e.g., `-- use_mapper` option can be enabled).

#### A.7 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-badging>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>



## REFERENCES

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. 265–283.
- [2] Jorge Albericio, Patrick Judd, Tayler Hetherington, Tor Aamodt, Natalie Enright Jerger, and Andreas Moshovos. 2016. Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*. 1–13. <https://doi.org/10.1109/ISCA.2016.11>
- [3] Ralph-Uwe Börner, Oliver G Ernst, and Stefan Güttel. 2015. Three-dimensional transient electromagnetic modelling using rational Krylov methods. *Geophysical Journal International (Geophys. J. Int)* 202, 3 (2015), 2025–2043.
- [4] Sofien Bouaziz, Andrea Tagliasacchi, and Mark Pauly. 2013. Sparse Iterative Closest Point. In *Proceedings of the 11th Eurographics/ACMSIGGRAPH Symposium on Geometry Processing (SGP)*. 113–123. <https://doi.org/10.1111/cgf.12178>
- [5] Aydin Buluç, Jeremy T. Fineman, Matteo Frigo, John R. Gilbert, and Charles E. Leiserson. 2009. Parallel Sparse Matrix-Vector and Matrix-Transpose-Vector Multiplication Using Compressed Sparse Blocks. In *Proceedings of the 21st Annual Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 233–244. <https://doi.org/10.1145/1583991.1584053>
- [6] Prasanth Chatarasi, Hyoukjun Kwon, Angshuman Parashar, Michael Pellauer, Tushar Krishna, and Vivek Sarkar. 2021. Marvel: A Data-Centric Approach for Mapping Deep Learning Operators on Spatial Accelerators. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 1, Article 6 (2021), 26 pages. <https://doi.org/10.1145/3485137>
- [7] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*. 367–379. <https://doi.org/10.1109/ISCA.2016.40>
- [8] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE Journal of Solid-State Circuits (JSSC)* 52, 1 (2017), 127–138. <https://doi.org/10.1109/JSSC.2016.2616357>
- [9] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems (JETCAS)* 9, 2 (2019), 292–308. <https://doi.org/10.1109/JETCAS.2019.2910232>
- [10] Jack Choquette, Edward Lee, Ronny Krashinsky, Vishnu Balan, and Bruce Khailany. 2021. The A100 Datacenter GPU and Ampere Architecture. In *Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. 48–50. <https://doi.org/10.1109/ISSCC42613.2021.9365803>
- [11] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proceedings of the ACM on Programming Languages (OOPSLA)* 2, Article 123 (2018), 30 pages.
- [12] Chunhua Deng, Yang Sui, Siyu Liao, Xuehai Qian, and Bo Yuan. 2021. GoSPA: An Energy-efficient High-performance Globally Optimized SParse Convolutional Neural Network Accelerator. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*. 1110–1123. <https://doi.org/10.1109/ISCA52012.2021.00090>
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805* (2018).
- [14] A. Einstein. 1916. The Foundation of the General Theory of Relativity. *Annalen der Physik* 354, 7 (1916), 769–822. <https://doi.org/10.1002/andp.19163540702>
- [15] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 151–165. <https://doi.org/10.1145/3352460.3358291>
- [16] Song Han, Huizi Mao, and William J Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *Proceedings of the International Conference on Learning Representations (ICLR)*. 1–14.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *arXiv: 1512.03385* (2015).
- [18] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 319–333.
- [19] Nathalie Henry, Jean-Daniel Fekete, and Michael J. McGuffin. 2007. NodeTriX: a Hybrid Visualization of Social Networks. *IEEE Transactions on Visualization and Computer Graphics (IEEE Trans Vis Comput Graph)* 13, 6 (2007), 1302–1309. <https://doi.org/10.1109/TVCG.2007.70582>
- [20] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv: 1704.04861* (2017).

- [21] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. In *Proceedings of the 48th Annual ACM/IEEE International Symposium on Computer Architecture (ISCA)*. 554–566. <https://doi.org/10.1109/ISCA52012.2021.00050>
- [22] Jun-Woo Jang, Sehwan Lee, Dongyoung Kim, Hyunsun Park, Ali Shafiee Ardestani, Yeongjae Choi, Channoh Kim, Yoojin Kim, Hyeongseok Yu, Hamzah Abdel-Aziz, Jun-Seok Park, Heonsoo Lee, Dongwoo Lee, Myeong Woo Kim, Hanwoong Jung, Heewoo Nam, Dongguen Lim, Seungwon Lee, Joon-Ho Song, Suknam Kwon, Joseph Hassoun, SukHwan Lim, and Changkyu Choi. 2021. Sparsity-Aware and Re-configurable NPU Architecture for Samsung Flagship Mobile SoC. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 15–28. <https://doi.org/10.1109/ISCA52012.2021.00011>
- [23] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *Proceedings of the IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [24] Liu Ke, Xin He, and Xuan Zhang. 2018. NNest: Early-Stage Design Space Exploration Tool for Neural Network Inference Accelerators. In *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED)*. Article 4, 6 pages. <https://doi.org/10.1145/3218603.3218647>
- [25] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. 473–486. <https://doi.org/10.1109/ISCA45697.2020.00047>
- [26] Fredrik Kjolstad, Stephen Chou, David Lugato, Shoaib Kamil, and Saman Amarasinghe. 2017. Taco: A tool to generate tensor algebra kernels. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 943–948. <https://doi.org/10.1109/ASE.2017.8115709>
- [27] Scott Kolodziej, Mohsen Mahmoudi Aznaveh, Matthew Bullock, Jarrett David, Timothy Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software (J. Open Source Softw.)* 4 (2019), 1–4. <https://doi.org/10.21105/joss.01244>
- [28] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Proceedings of Advances in Neural Information Processing Systems (NIPS)*, Vol. 25.
- [29] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar. 2020. MAESTRO: A Data-Centric Approach to Understand Reuse, Performance, and Hardware Cost of DNN Mappings. *IEEE Micro* 40, 3 (2020), 20–29.
- [30] Francisco Muñoz Matrínez, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2021. STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators. *IEEE Computer Architecture Letters (CAL)* 01 (2021).
- [31] Linyan Mei, Pouya Houshmand, Vikram Jain, Juan Sebastian P. Giraldo, and Marian Verhelst. 2020. ZigZag: A Memory-Centric Rapid DNN Accelerator Design Space Exploration Framework. *arxiv:2007.11360* (2020).
- [32] M. Motamedi, P. Gysel, V. Akella, and S. Ghiasi. 2016. Design space exploration of FPGA-based Deep Convolutional Neural Networks. In *Proceedings of the 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. 575–580.
- [33] M. Nikolić, M. Mahmoud, and A. Moshovos. 2018. Characterizing Sources of Ineffectual Computations in Deep Learning Networks. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*. 86–87. <https://doi.org/10.1109/IISWC.2018.8573509>
- [34] NVIDIA. 2020. *NVIDIA A100 Tensor Core GPU Architecture*. Technical Report. NVIDIA.
- [35] S. Pal, J. Beaumont, D. Park, A. Amarnath, S. Feng, C. Chakrabarti, H. Kim, D. Blaauw, T. Mudge, and R. Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*. 724–736. <https://doi.org/10.1109/HPCA.2018.00067>
- [36] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315. <https://doi.org/10.1109/ISPASS.2019.00042>
- [37] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W. Keckler, and William J. Dally. 2017. SCNN: An Accelerator for Compressed-Sparse Convolutional Neural Networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. 27–40. <https://doi.org/10.1145/3079856.3080254>
- [38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Proceedings of Advances in Neural Information Processing Systems (NIPS)*. 8024–8035.
- [39] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. <https://doi.org/10.1109/HPCA47549.2020.00015>
- [40] A. Rahman, S. Oh, J. Lee, and K. Choi. 2017. Design space exploration of FPGA accelerators for convolutional neural networks. In *Proceedings of the Design, Automation Test in Europe Conference Exhibition (DATE)*. 1147–1152.
- [41] Chiara Ravazzi, Roberto Tempo, and Fabrizio Dabbene. 2018. Learning Influence Structure in Sparse Social Networks. *IEEE Transactions on Control of Network Systems* 5, 4 (2018), 1976–1986. <https://doi.org/10.1109/TCNS.2017.2781367>

- [42] Yousef Saad. 2011. *Numerical Methods for Large Eigenvalue Problems*. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611970739>
- [43] Ananda Samajdar, Jan Moritz Joseph, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2020. A Systematic Methodology for Characterizing Scalability of DNN Accelerators using SCALE-Sim. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 58–68. <https://doi.org/10.1109/ISPASS48437.2020.00016>
- [44] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra, and Hadi Esmaeilzadeh. 2016. From high-level deep neural models to FPGAs. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. <https://doi.org/10.1109/MICRO.2016.7783720>
- [45] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556* (2015).
- [46] Shaden Smith and George Karypis. 2015. Tensor-Matrix Products with a Compressed Sparse Tensor. In *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. Article 5, 7 pages. <https://doi.org/10.1145/2833179.2833183>
- [47] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonese, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 766–780. <https://doi.org/10.1109/MICRO50266.2020.00068>
- [48] Nitish Srivastava, Hongbo Rong, Prithayan Barua, Guanyu Feng, Huanqi Cao, Zhiru Zhang, David Albonese, Vivek Sarkar, Wenguang Chen, Paul Petersen, Geoff Lowney, Adam Herr, Christopher Hughes, Timothy Mattson, and Pradeep Dubey. 2019. T2S-Tensor: Productively Generating High-Performance Spatial Hardware for Dense Tensor Computations. In *Proceedings of the 27th Annual IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 181–189. <https://doi.org/10.1109/FCCM.2019.00033>
- [49] Wei Sun, Ang Li, Tong Geng, Sander Stuijk, and Henk Corporaal. 2022. Dissecting Tensor Cores via Microbenchmarks: Latency, Throughput and Numerical Behaviors. *arXiv: 12206.02874* (2022).
- [50] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2020. Efficient Processing of Deep Neural Networks. *Synthesis Lectures on Computer Architecture* 15, 2 (2020), 1–341. <https://doi.org/10.2200/S01004ED1V01Y202004CAC050>
- [51] The SciPy community. [n. d.]. SciPy documentation. [https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.coo\\_matrix.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.coo_matrix.html)
- [52] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. Emer, S. W. Keckler, and B. Khailany. 2019. MAGNet: A Modular Accelerator Generator for Neural Networks. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8.
- [53] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-Side Sparse Tensor Core. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*. 1083–1095. <https://doi.org/10.1109/ISCA52012.2021.00088>
- [54] Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2019. Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. <https://doi.org/10.1109/ICCAD45719.2019.8942149>
- [55] Yannan Nellie Wu, Vivienne Sze, and Joel S. Emer. 2020. An Architecture-Level Energy and Area Estimator for Processing-In-Memory Accelerator Designs. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 116–118. <https://doi.org/10.1109/ISPASS48437.2020.00024>
- [56] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. [n. d.]. Sparseloop website. <http://sparseloop.mit.edu/>
- [57] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. [n. d.]. Timeloop code base. <https://github.com/NVlabs/timeloop>
- [58] Pengfei Xu, Xiaofan Zhang, Cong Hao, Yang Zhao, Yongan Zhang, Yue Wang, Chaojian Li, Zetong Guan, Deming Chen, and Yingyan Lin. 2020. AutoDNNchip: An Automated DNN Chip Predictor and Builder for Both FPGAs and ASICs. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 40–50. <https://doi.org/10.1145/3373087.3375306>
- [59] Dingqing Yang, Amin Ghasemazar, Xiaowei Ren, Maximilian Golub, Guy Lemieux, and Mieszko Lis. 2020. Procrustes: a Dataflow and Accelerator for Sparse Deep Neural Network Training. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 711–724. <https://doi.org/10.1109/MICRO50266.2020.00064>
- [60] Tien-Ju Yang, Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2017. A method to estimate the energy consumption of deep neural networks. In *Proceedings of the 51st Asilomar Conference on Signals, Systems, and Computers (Asilomar)*. 1916–1920. <https://doi.org/10.1109/ACSSC.2017.8335698>
- [61] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson’s Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 687–701. <https://doi.org/10.1145/3445814.3446702>
- [62] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, and Yunji Chen. 2016. Cambricon-X: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. <https://doi.org/10.1109/MICRO.2016.7783723>

- [63] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W. Hwu, and D. Chen. 2018. DNNBuilder: an Automated Tool for Building High-Performance DNN Hardware Accelerators for FPGAs. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8.
- [64] Z. Zhang, H. Wang, S. Han, and W. J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–274. <https://doi.org/10.1109/HPCA47549.2020.00030>
- [65] M. Zhao, R.V. Panda, S.S. Sapatnekar, and D. Blaauw. 2002. Hierarchical analysis of power distribution networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 21, 2 (2002), 159–168. <https://doi.org/10.1109/43.980256>
- [66] Yang Zhao, Chaojian Li, Yue Wang, Pengfei Xu, Yongan Zhang, and Yingyan Lin. 2020. DNN-Chip Predictor: An Analytical Performance Predictor for DNN Accelerators with Various Dataflows and Hardware Architectures. In *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*. 1593–1597. <https://doi.org/10.1109/ICASSP40776.2020.9053977>
- [67] Maohua Zhu, Tao Zhang, Zhenyu Gu, and Yuan Xie. 2019. Sparse Tensor Core: Algorithm and Hardware Co-Design for Vector-Wise Sparse Neural Networks on Modern GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 359–371. <https://doi.org/10.1145/3352460.3358269>